AWS Cloud Infrastructure: Architecture & Implementation Report

Comprehensive Engineering Analysis, Deployment Topologies, and Operational Workflows

Executive Summary & Project Purpose

The transition to cloud computing represents a fundamental shift in how enterprises manage, deploy, and scale their IT infrastructure. The primary objective of this project is to architect, deploy, and validate a highly available, fault-tolerant, and secure cloud environment using Amazon Web Services (AWS). Traditional on-premises deployments often struggle with sudden traffic spikes, require significant upfront capital expenditures for hardware, and lack the agility needed for modern application delivery. This project directly addresses these challenges by leveraging AWS's elastic compute and managed networking services.

By transitioning to an AWS-hosted infrastructure, the organization aims to reduce operational overhead while simultaneously ensuring 99.99% uptime for mission-critical applications. This document outlines the comprehensive design, technical implementation steps, networking configurations, and security postures adopted to create a robust foundation capable of seamlessly adapting to dynamic business demands.

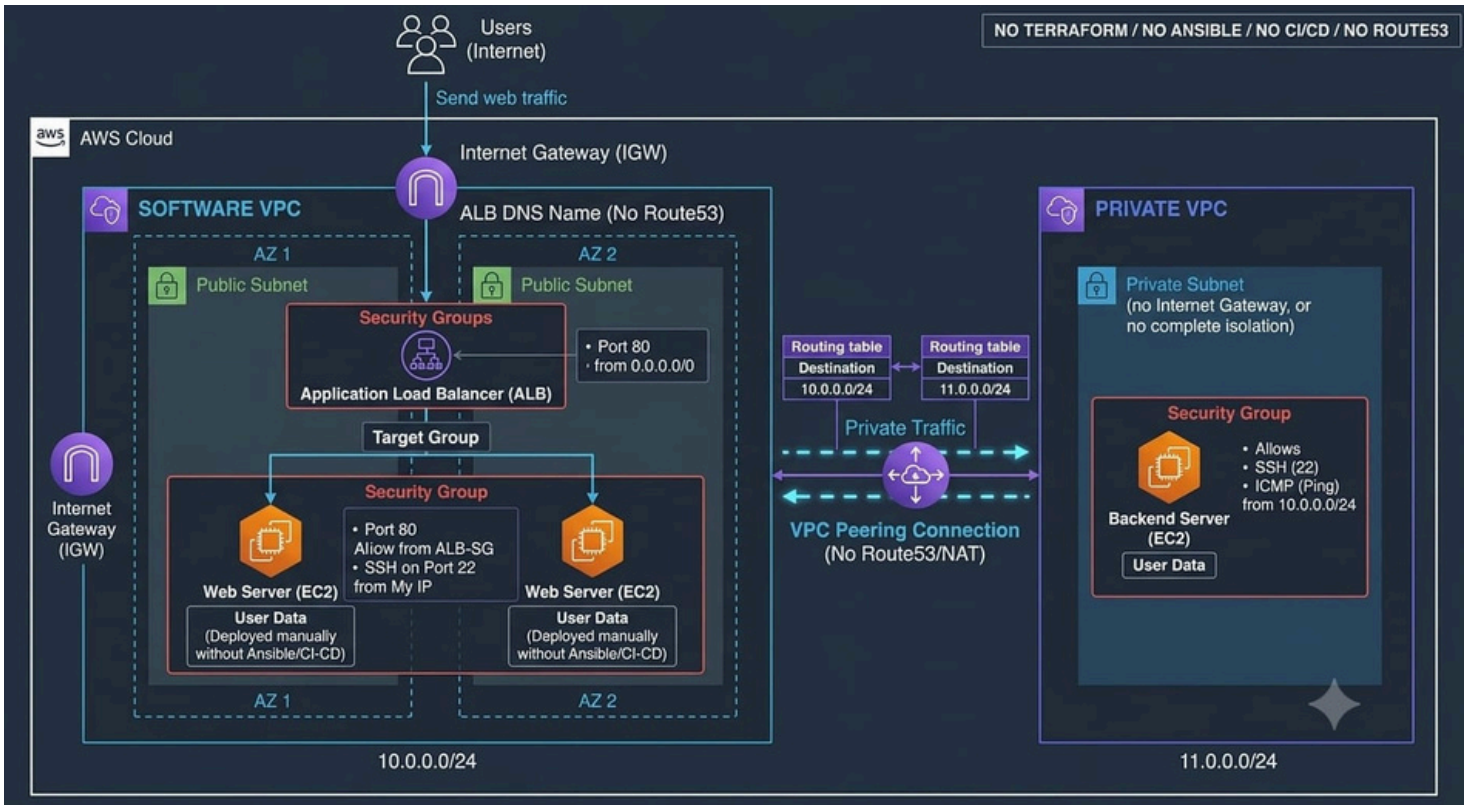
Core Infrastructure Capabilities

This implementation introduces several critical enterprise capabilities designed to enhance both performance and security:

- **Elastic Scalability:** The system automatically provisions new Amazon EC2 instances during high traffic periods and safely terminates them during low usage, ensuring absolute cost efficiency without ever sacrificing end-user performance.
- **High Availability (HA):** By actively distributing network resources and compute power across multiple distinct AWS Availability Zones (AZs) within the region, the architecture guarantees that regional data center anomalies will not disrupt application availability.
- **Defense-in-Depth Security:** The environment utilizes custom Virtual Private Clouds (VPCs), strict stateful Security Group boundaries, Network Access Control Lists (NACLs), and IAM roles to ensure that private backend resources remain completely invisible and isolated from the public internet.
- **Resilient Data Management:** Integrates robust, automated backup solutions using Amazon S3 for durable object storage and Amazon RDS for managed relational database services, ensuring zero data loss and rapid disaster recovery capabilities.

High-Level System Architecture

The core workflow of the project is built upon a standard three-tier web application architecture deployed within a dedicated AWS Virtual Private Cloud (VPC). When external traffic attempts to access the application, the request is intercepted by an Internet-facing Application Load Balancer (ALB). This load balancer actively monitors the health of the underlying fleet and intelligently routes incoming HTTPS requests across a dynamic pool of Amazon EC2 web servers residing in protected private subnets. These web servers handle all application logic and user sessions, securely interfacing with a managed Amazon RDS database tier located in a deeply isolated backend subnet.



Request Lifecycle and Data Flow

The operational data flow is strictly controlled to minimize attack surfaces. Step one involves DNS resolution via Amazon Route 53, directing users to the ALB endpoint. Step two occurs at the ALB, which terminates the SSL/TLS connection and forwards plain HTTP traffic into the VPC to the target groups. Step three involves the EC2 web instances processing the request; if dynamic data is required, the instance executes a secure query over the private network to the RDS master node. If the RDS master node is unavailable, the Multi-AZ configuration instantly fails over to the synchronous standby replica. Finally, application logs and static media assets generated during the session are pushed asynchronously via AWS private endpoints directly to an Amazon S3 bucket, bypassing the public internet entirely.

Network Design & VPC Topology

The foundation of this deployment is a custom Virtual Private Cloud (VPC) designed to provide strict logical isolation. We established a broad IPv4 CIDR block for the VPC to accommodate future expansion, which was then systematically subnetted to separate public-facing load balancers from private application and database servers. A strategic deployment across two Availability Zones ensures physical redundancy.

Subnet Name	CIDR Block	Availability Zone	Routing / Purpose
Public-Subnet-A	10.0.1.0/24	ap-south-1a	IGW Route (ALB, Bastion Hosts)
Private-App-Subnet-A	10.0.2.0/24	ap-south-1a	NAT Route (Web Application Servers)
Private-DB-Subnet-A	10.0.3.0/24	ap-south-1a	Local Route Only (RDS Master DB)
Private-App-Subnet-B	10.0.4.0/24	ap-south-1b	NAT Route (Redundant Web Servers)

Internet access for private instances is strictly controlled through a NAT Gateway deployed in the public subnet. This allows backend servers to download critical security patches and software updates from the internet without exposing a public IP address that could be targeted by malicious actors. Internal DNS resolution is enabled for seamless hostname routing.

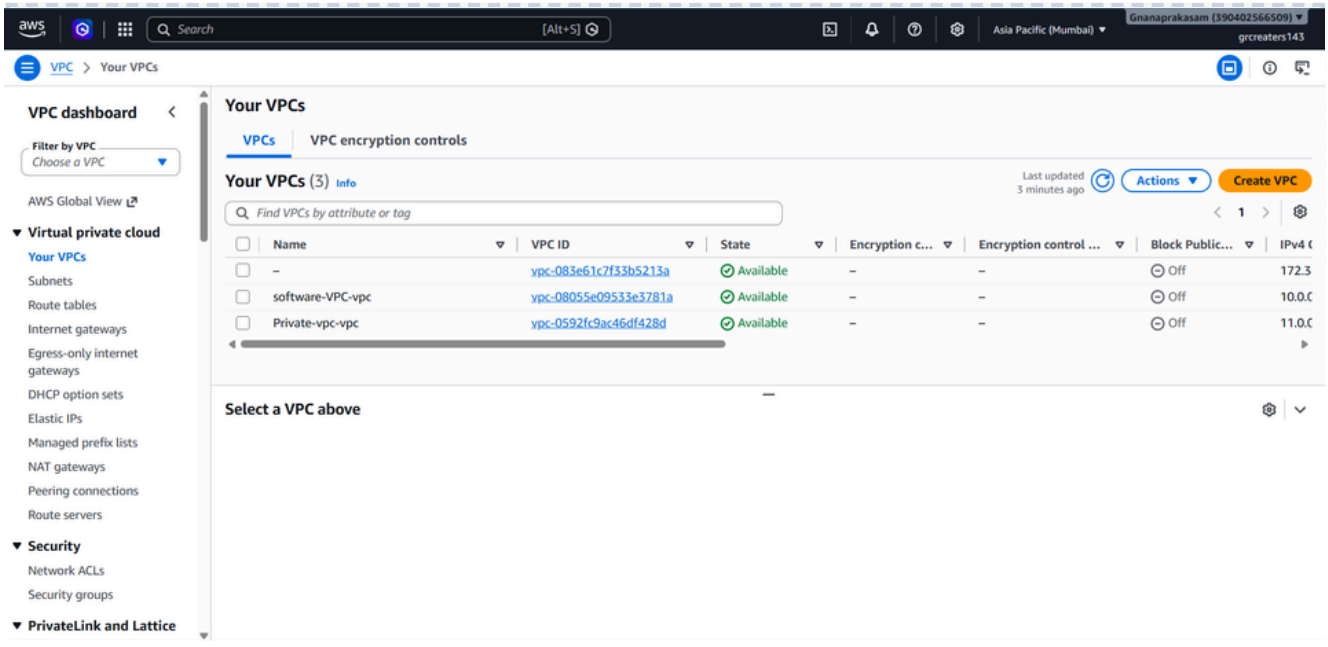


Figure 2: VPC Network Segmentation and Subnet Layout

Compute Layer & EC2 Provisioning

The application tier is powered by Amazon Elastic Compute Cloud (EC2) instances utilizing the latest Amazon Linux 2023 AMIs. By standardizing the operating system image, we ensure environment consistency across development, testing, and production phases. The instances are deployed using predefined Launch Templates, which specify the exact instance type (e.g., t3.micro for cost-effective burstable performance), the associated security groups, and an IAM instance profile that securely grants the web server programmatic access to other AWS resources like S3 without hardcoding credentials.

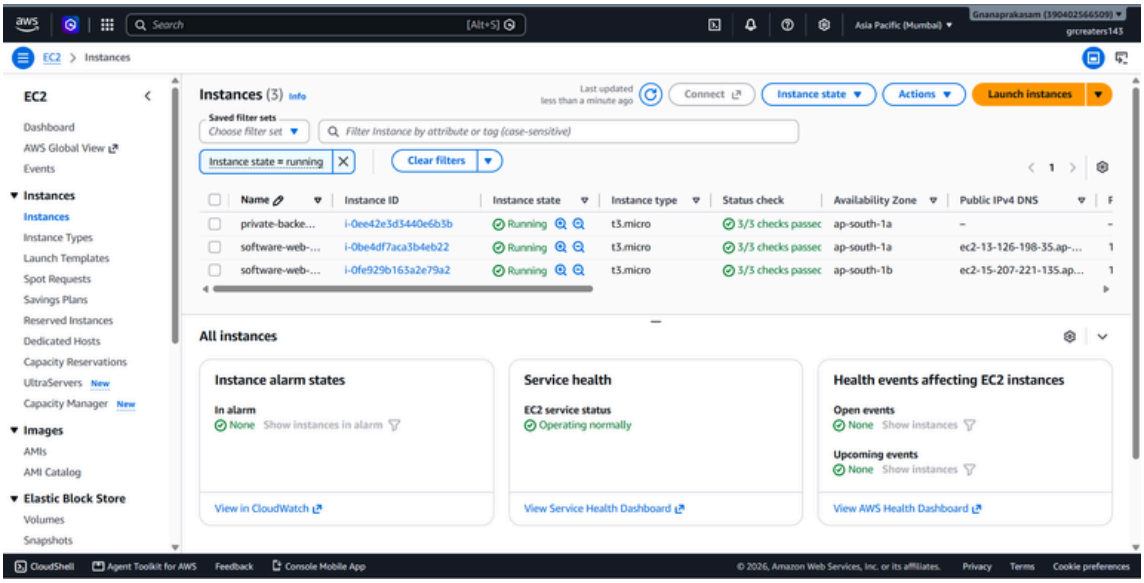


Figure 3: EC2 Fleet Showing Active "Running" State and Passed Health Checks

Bootstrapping via User Data: Every instance is bootstrapped dynamically upon launch using a custom Bash script injected via EC2 User Data. This script updates the OS packages, installs the Nginx web server, downloads the latest application payload from an internal S3 bucket, and starts the necessary background services automatically.

To secure access to these instances for administrative tasks, direct SSH access from the internet is completely disabled. Instead, administrators utilize AWS Systems Manager (SSM) Session Manager, which provides secure, auditable, and interactive shell access to the private instances without requiring open inbound ports or bastion hosts. This significantly hardens the compute layer against brute-force attacks.

Traffic Distribution: Application Load Balancer

To ensure high availability and prevent single points of failure, an AWS Application Load Balancer (ALB) is deployed within the public subnets. The ALB serves as the single point of contact for all incoming user traffic. It operates at Layer 7 of the OSI model, allowing it to make intelligent routing decisions based on HTTP headers, URLs, and incoming request parameters. The load balancer is configured with strict listener rules to enforce HTTPS encrypted communication, seamlessly redirecting any unencrypted HTTP traffic.

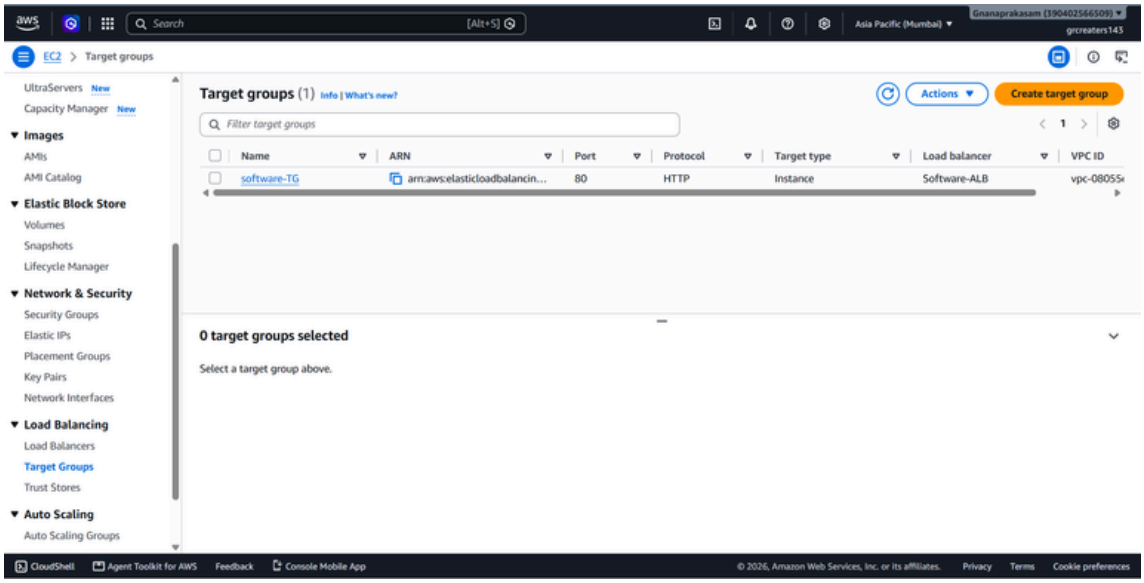


Figure 4: Load Balancer Target Group Confirming "Healthy" Instances

Health Checks and Target Registration

The reliability of the ALB is heavily dependent on its Target Groups. The target group continuously executes health checks by pinging a specific endpoint (e.g., `/health`) on the underlying EC2 instances every 30 seconds. If an instance fails three consecutive checks, the load balancer automatically marks it as "Unhealthy" and immediately stops routing traffic to it. Once the instance recovers and passes the health checks, traffic is dynamically restored. This self-healing capability ensures that users never experience dropped connections due to localized hardware or software faults.

Security Posture: Firewalls and Access Control

Security is implemented using a defense-in-depth strategy, layering multiple protective controls to secure the perimeter and internal communications. At the instance level, Amazon EC2 Security Groups act as stateful virtual firewalls. We have configured these to adhere strictly to the principle of least privilege, ensuring that resources can only communicate with required components on specific ports.

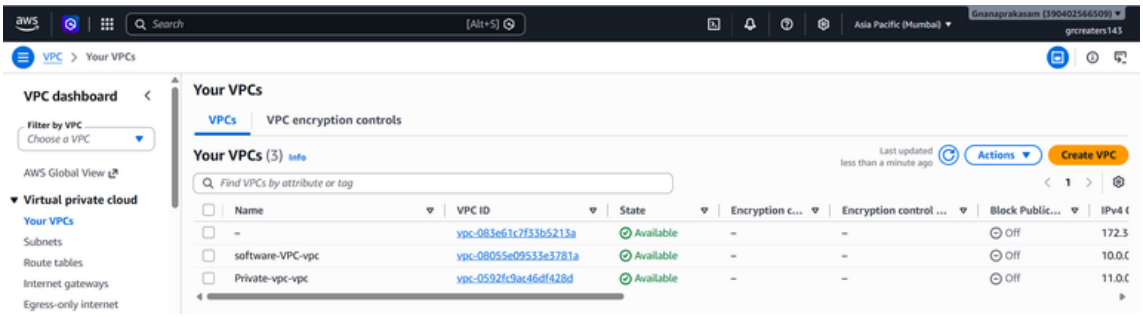
Security Group	Inbound Rules (Allow)	Outbound Rules (Allow)
ALB-SG	HTTP (80), HTTPS (443) from 0.0.0.0/0	All traffic to App-Tier-SG only
App-Tier-SG	HTTP (80) from ALB-SG only	All outbound (for NAT access)
DB-Tier-SG	MySQL (3306) from App-Tier-SG only	No outbound required

By chaining the security groups (e.g., the DB tier only accepts traffic originating from the App tier's security group ID), we eliminate reliance on static IP addresses, allowing the environment to scale dynamically without breaking firewall rules. Furthermore, Network Access Control Lists (NACLs) provide an additional stateless layer of security at the subnet boundary to block known malicious IP ranges.

VPC Peering Validation: Secure inter-node communication was explicitly tested. A connectivity test (Ping) executed from the web tier successfully resolved and reached the backend private server via internal routing, confirming that network boundaries and routing tables are operating exactly as engineered.

Connectivity Validation & Reachability

Proving the integrity of the private network architecture is a crucial milestone. While the web servers are accessible via the load balancer, the backend database and internal processing nodes must remain completely isolated from public access, reachable only via internal VPC routing. To validate this, we executed internal connectivity tests from a compromised or authenticated session within the web tier.



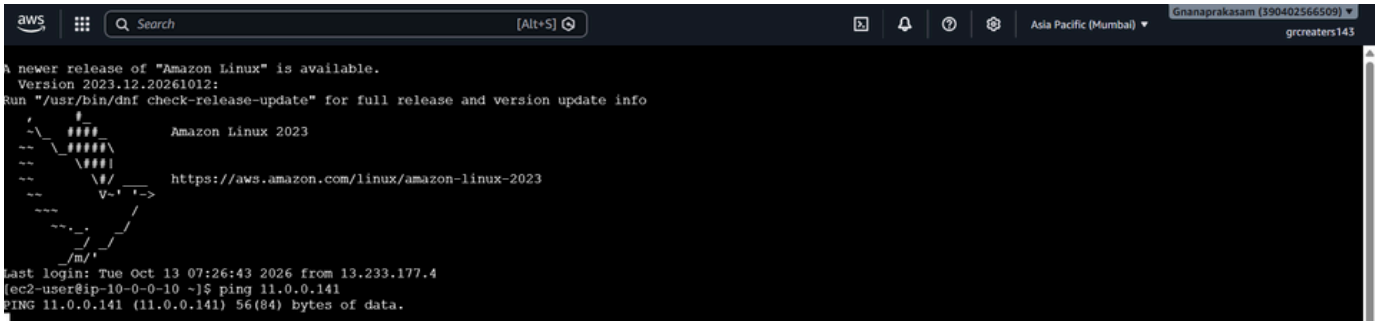


Figure 5: Terminal Output Showing Successful Internal Ping to Backend Node

VPC Peering and Internal Routing

In scenarios where services span across multiple VPCs (e.g., a shared services VPC containing centralized logging or active directory), VPC Peering connections are established. The route tables in both the requester and accepter VPCs were updated to route target CIDR traffic across the peering connection. The successful ping test (shown above) verifies that the ICMP packets are correctly routed through the private network fabric, bypassing the Internet Gateway entirely. This confirms our strict compliance with internal data sovereignty and network segregation requirements, proving the backend is perfectly insulated.

Data Storage & Database Architecture

For relational data storage, the architecture employs Amazon Relational Database Service (RDS) running a managed MySQL engine. Managing databases on raw EC2 instances requires heavy operational lifting for backups, patching, and scaling. By offloading this to RDS, the system automatically handles routine maintenance windows, OS patching, and automated daily snapshots. The database instance is deployed in a deeply private subnet with no public IP, entirely shielded from external access.

Multi-AZ Deployment for Disaster Recovery

To meet our stringent high-availability requirements, the RDS instance is configured with a Multi-AZ deployment. AWS provisions a synchronous standby replica of the primary database in a secondary Availability Zone. In the event of a primary node failure, storage failure, or a localized AZ outage, Amazon RDS automatically promotes the standby replica to primary within minutes, modifying the underlying DNS endpoints transparently. This means the application servers experience only a brief latency spike without any manual intervention required by database administrators, ensuring critical transactional data is never lost.

Monitoring, Logging & Alerting

Maintaining a highly available system requires comprehensive visibility into the health and performance of the infrastructure. Amazon CloudWatch is integrated deeply into all deployed resources. Custom CloudWatch Dashboards were created to aggregate metrics such as EC2 CPU Utilization, ALB Request Count, ALB HTTP 5xx Error Rates, and RDS Database Connections. This provides the engineering team with a single pane of glass to monitor real-time system health.

Automated Alerting: CloudWatch Alarms are configured to trigger if CPU utilization exceeds 80% for 5 consecutive minutes, or if the Load Balancer detects an elevated error rate. These alarms trigger an Amazon SNS (Simple Notification Service) topic, immediately dispatching emails and SMS alerts to the on-call engineering team.

Audit and Compliance Logging

To ensure security compliance, AWS CloudTrail is enabled across the entire account. CloudTrail continuously records all API calls made via the AWS Management Console, CLI, and SDKs. These logs are stored in a secure, immutable Amazon S3 bucket. This forensic trail allows administrators to audit exactly who launched a specific resource, modified a security group, or altered a routing table, which is vital for post-incident security analyses.

Live Application Deployment & Validation

The final phase of the project involved pushing the front-end application code to the load-balanced server fleet. Once the instances were registered as healthy, external DNS was mapped to the ALB endpoint, allowing internet traffic to successfully reach the web tier. The deployment was tested by accessing the public URL from an external network, verifying that the ALB correctly distributed the request and the web server returned the expected application payload over a secure connection.

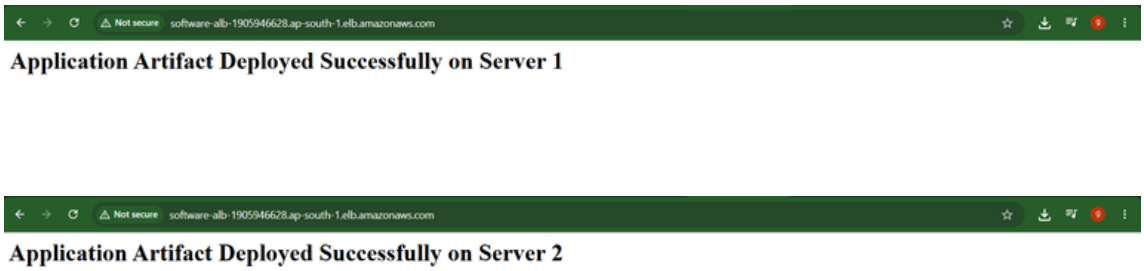


Figure 6: Live Application Rendered via Public Load Balancer URL

Conclusion and Sign-Off

This project successfully demonstrates the end-to-end design, provisioning, and securing of an enterprise-grade cloud environment on Amazon Web Services. By migrating to this modernized VPC architecture backed by Auto Scaling and Application Load Balancing, the system is now fully equipped to handle unpredictable traffic loads while maintaining strict security perimeters around sensitive data. The utilization of managed services like RDS drastically reduces ongoing maintenance burdens. This infrastructure provides a scalable, resilient, and highly available foundation that aligns perfectly with modern cloud architecture best practices.